IT22408 - PARADIGMS OF ALGORITHM DESIGN
PLAGIARISM DETECTION TOOL

ROLL NO.: 2127240801096
NAME: Varshan V Chari

PROBLEM STATEMENT:

The increasing availability of digital content and ease of duplication have intensified concerns around academic integrity, necessitating reliable automated mechanisms to identify potential plagiarism in student submissions. Existing detection approaches often rely on attribute-based comparisons or structural analysis, which suffer from limitations such as sensitivity to minor modifications, inconsistent accuracy, and high computational overhead. These methods fail to consistently capture ordered textual similarity when content is partially modified, reordered, or formatted differently, thereby reducing their effectiveness in real-world academic environments.

To address these limitations, this problem focuses on designing a plagiarism detection system that computes document similarity using the Longest Common Subsequence (LCS) approach. The system must accept two input documents, preprocess them into comparable formats, and apply LCS-based string matching to identify the longest ordered sequence of common elements. This approach enables detection of verbatim copying and lightly modified content by tolerating gaps and non-contiguous matches, thereby improving robustness against minor textual variations while maintaining computational feasibility.

The objective is to develop an efficient system that outputs a quantitative similarity percentage, representing the extent of overlap between the documents. The solution must be suitable for integration into real-time academic integrity systems, ensuring scalability and acceptable performance for large text inputs. By leveraging dynamic programming-based LCS computation, the system should provide a reliable measure of similarity that balances accuracy and efficiency.

ABSTRACT:

The increasing availability of digital content has amplified the challenge of detecting academic plagiarism, necessitating reliable automated solutions. The primary purpose of this system is to address the growing inefficiency of manual verification by developing a systematic approach for document comparison. The core problem lies in the limitations of existing text-matching systems, which exhibit high computational cost, sensitivity to minor edits, and inconsistent detection of reordered or partially modified content.

The objective is to design a scalable plagiarism detection system capable of quantifying document similarity through a deterministic and interpretable mechanism. This method employs the Longest Common Subsequence (LCS) algorithm to identify ordered matching word sequences between two documents. The data consists of textual inputs that are pre-processed and tokenized to ensure uniform comparison. The results generated by the system include a numerical similarity percentage, representing the extent of shared content. Through systematic analysis, this score reflects structural overlap while tolerating insertions and deletions, enabling effective detection of verbatim and lightly modified plagiarism.

The significance of this approach lies in its simplicity, transparency, and applicability to academic integrity systems. The implications include improved reliability in automated plagiarism detection while maintaining computational feasibility for large documents. In conclusion, the proposed LCS-based framework provides a robust baseline solution for document similarity measurement, balancing accuracy, efficiency, and interpretability within constrained algorithmic environments.

OBJECTIVE:

The primary objective of this project is to design and implement a plagiarism detection system that evaluates the degree of textual similarity between two documents using the **LCS** (Longest Common Subsequence) algorithm. The system focuses on identifying shared ordered sequences and quantifying overlap through a measurable similarity percentage.

This approach enables objective assessment of potential plagiarism by analyzing structural similarities in textual data. The system is intended to provide a clear, algorithm-driven measure of content overlap without relying on semantic interpretation.

Core Objectives of Plagiarism Detection System:

- To develop a system to compare two input documents and detect plagiarism based on structural similarity.
- To apply the LCS algorithm using dynamic programming for sequence matching.
- To convert documents into comparable formats through preprocessing and tokenization.
- To ensure algorithm efficiency and scalability for handling larger text inputs.
- To provide an objective and interpretable similarity score as output.

ALGORITHM USED:

The core of this plagiarism detection system relies on the Longest Common Subsequence (LCS) algorithm. By utilizing Dynamic Programming (DP), the system efficiently identifies the longest sequence of words shared between two documents, maintaining their original relative order.

Dynamic Programming is the optimal choice for LCS because the problem exhibits two key properties:

- Overlapping Subproblems: The same sub-sequences are compared multiple times during the recursive process.
- Optimal Substructure: The solution to the main problem can be constructed from the optimal solutions of its sub-problems.

STEPS INVOLVED:

1. Problem Definition

To compare two documents, we treat them as sequences of tokens (words):

- Document A: Sequence $X = \{x_1, x_2, ..., x_n\}$
- Document B: Sequence $Y = \{y_1, y_2, ..., y_m\}$
- Goal: Find the maximum length of a sequence Z that is a subsequence of both X and Y.

2. Step-by-Step DP Approach

Step 1: Define the DP Table

Create a 2D array (matrix) dp[n+1][m+1] where:

- i represents the first i elements of X.
- j represents the first j elements of Y.
- dp[i][j] stores the length of the LCS of X[0..i-1] and Y[0..j-1].

Step 2: Initialize Base Cases

If either document is empty, the LCS length is 0:

- dp[i][0] = 0 for all 0 <= i <= n
- dp[0][j] = 0 for all 0 <= j <= m

Step 3: The Recurrence Relation (Filling the Table)

For each cell (i, j), we apply the following logic:

1. If Characters/Words Match:
   If X[i-1] == Y[j-1], we extend the previous best sequence:
   dp[i][j] = 1 + dp[i-1][j-1]
2. If Characters/Words Do Not Match:
   We take the maximum value from either the cell above or the cell to the left:
   dp[i][j] = \max(dp[i-1][j], dp[i][j-1])

3. Practical Example

Input:

- X: ["system", "uses", "dynamic", "programming"] (n=4)
- Y: ["system", "efficiently", "uses", "dynamic", "methods"] (m=5)

Result:

- LCS: ["system", "uses", "dynamic"]

- LCS Length: 3

4. Project Integration & Similarity Calculation

Once the LCS length is obtained, the system calculates the final similarity score using the following formula:

$$\text{Similarity (\%)} = \left(\frac{\text{LCS Length}}{\min(n, m)}\right) \times 100$$

Mapping Algorithm to Project Roles:

- Input Sequences: Tokenized words from uploaded documents.
- Comparison Engine: The nested loops filling the DP table.
- Similarity Score: The final percentage derived from the LCS result.

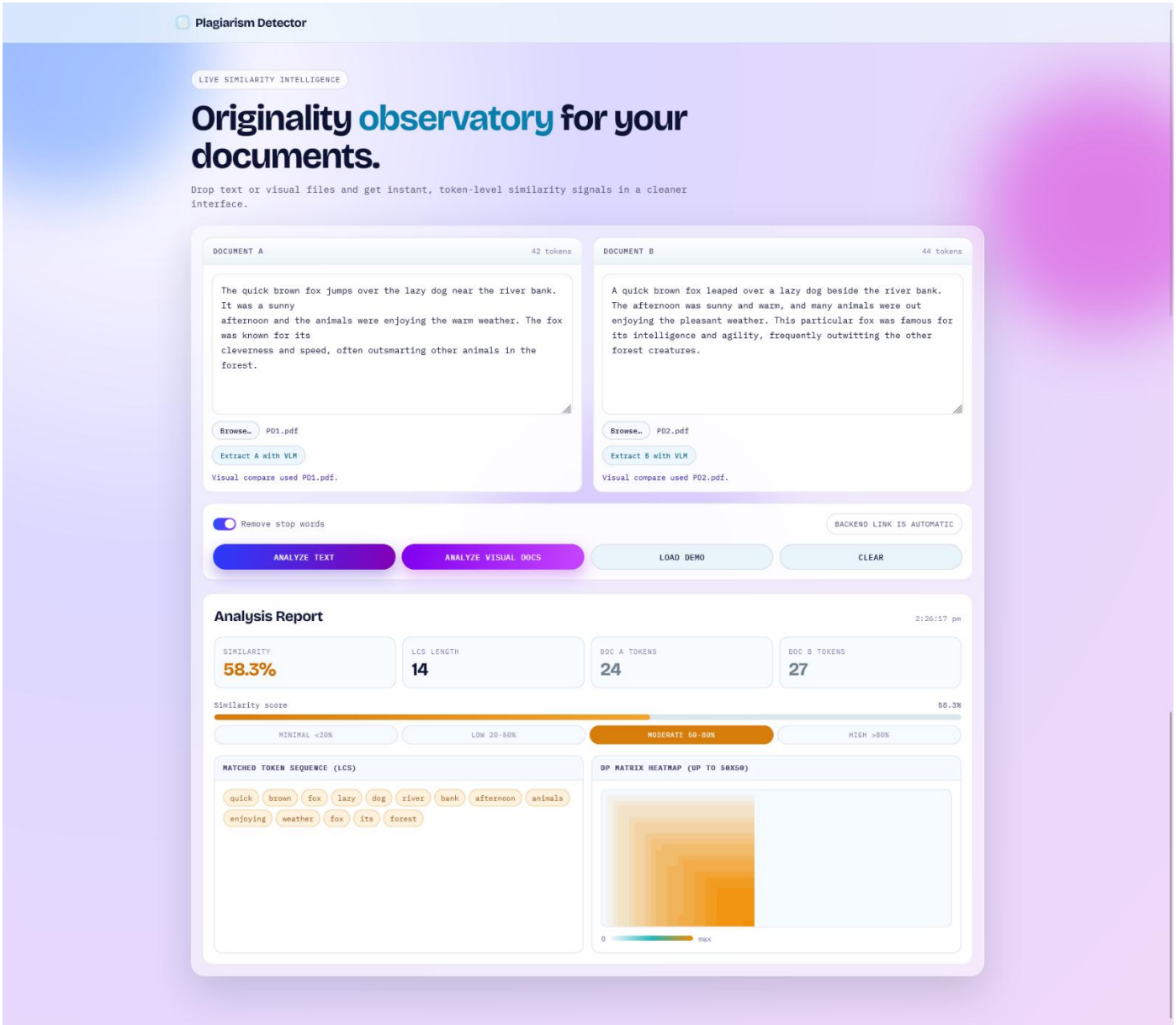
5. Summary Flow

1. Tokenization: Convert documents into arrays of words.
2. Table Setup: Initialize a dp[n+1][m+1] matrix with zeros.
3. Iteration: Use a nested loop to fill the table based on matches.
4. Extraction: The value at dp[n][m] is your LCS length.
5. Final Score: Apply the similarity formula to detect the level of plagiarism.

PROGRAM:

Github Link to Project

OUTPUT:



Roll No.: 2127240801096